マルチエージェントオーケストレーション入門

uma-chan

2026-03-05

1. はじめに

1.1. はじめに一言

タイトルに「入門」とつけたが、正直に言う

- マルチエージェントオーケストレーションは概念として難しい
- 基盤となる tmux も、慣れていない人には難しい
- 「入門と書いてある本あるある」の罠にハマっているかもしれない、ごめんなさい

今日のゴール: この資料を読んで「マルチエージェントオーケストレーション、いいな」と思う人が一人でも増えること

1.2. tmux 記事からの続き

「人間 1 人が 1 つのコーディングエージェントに依頼して完結させる」やり方では得られない体験がある — それを得るためにマルチエージェントオーケストレーションを考える

今回のテーマの布石として、以前 Slack でシェアした tmux 活用記事がある

- tmux のウインドウ・ペイン分割で複数の AI エージェントを並べて管理できる
- ペインタイトルをノード名として使うことで、エージェントの役割を明示できる
- 外部プロセスからの CLI 制御・ペイン命名が、エージェントルーティングの基盤になる
- 記事: https://zenn.dev/i9wa4/articles/2026-02-08-tmux-intro-ai-agent-orchestration

その記事の 7.3 節で次のように予告した

> 次回はこの tmux 利用方法をある程度取り入れていれば簡単に実現できるマルチエージェントオーケストレーションの構築方法を紹介します

今日はその続きとして、エコシステムの全体像を整理しながらマルチエージェントの考え方を深掘りする

1.3. 1:1 モデルのメリット・デメリット

「1 人の人間と 1 つのエージェント」という構成を正直に評価する

メリット

- セットアップ不要、すぐ使える
- コンテキストが一箇所に集まり管理が楽

デメリット

- コンテキストウインドウの上限に当たりやすい
- 長時間作業で判断精度が落ちる
- 一つの視点しか得られない（批判的検討が弱い）
- 人間の介入タイミングが不明確

2. コンテキストウインドウという制約

2.1. このセクションで引用する研究

このセクションで引用する研究

arXiv:2511.15755 — Philip Drammeh (2024)

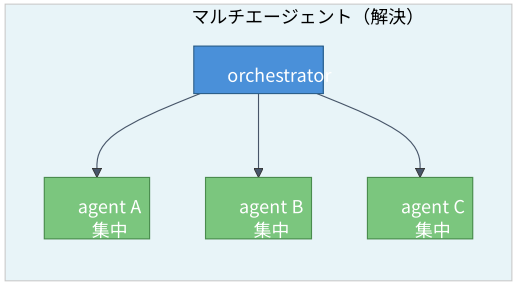
Multi-Agent LLM Orchestration Achieves Deterministic, High-Quality Decision Support for Incident Response

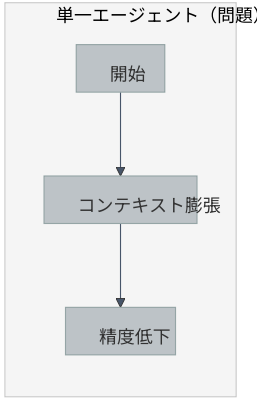
348 件の対照実験でマルチエージェントと単一エージェントを比較し、エージェントを専門化・分割するほど推奨の質と確実性が飛躍的に向上することを定量的に示した。

2.2. なぜコンテキストが足りないのか

AI コーディングエージェントを使い続けると必ずぶつかる壁

- コンテキストウインドウには上限がある（例: Claude Sonnet で 200K トークン）
- 長時間の作業・大規模コードベースでは会話履歴が圧迫される
- 圧縮・切り捨てが発生すると、エージェントの判断精度が下がる





コンテキストは「作業メモリ」

- 一人の人間に長大な作業を一気にやらせるようなもの
- 分割・委譲によって各エージェントのコンテキストを小さく保つことが鍵

前述の研究によると (arXiv:2511.15755)

- 50-100 トークンの短い専門プロンプト vs. 200+ トークンの複雑なプロンプト — 短い方が品質安定

3. マルチエージェントフレームワーク概観

3.1. 主要フレームワーク比較

現在の主要フレームワークと特徴

フレームワーク / ツール	主な特徴	GitHub URL (Stars)
LangGraph	グラフベース・永続状態・LangSmith 観測	https://github.com/langchain-ai/langgraph
AutoGen (Microsoft)	会話型・イベント駆動	https://github.com/microsoft/autogen (55,000+ stars)
CrewAI	ロールベース・高速開発	https://github.com/crewAIInc/crewAI (45,000+ stars)
OpenHands	AIコーディングエージェント・SWE-Bench 77.6%	https://github.com/All-Hands-AI/OpenHands (68,000+ stars)
MetaGPT	作業手順書（SOP: Standard Operating Procedures）駆動・ソフトウェア会社シミュレーション	https://github.com/FoundationAgents/MetaGPT (64,000+ stars)
Swarm (OpenAI)	教育用・軽量・OpenAI Agents SDK に移行済み	https://github.com/openai/swarm (~21,000 stars)
Agency Swarm	組織構造モデル・OpenAI Agents SDK	https://github.com/VRSEN/age

3.2. フレームワーク選択の視点

どのフレームワークも「コンテキスト管理」という共通問題に取り組んでいる

アプローチ別の整理

アプローチ	代表例	コンテキスト管理の方法
グラフ型	LangGraph	中央集権的な共有ステート
会話型	AutoGen	エージェント間対話で調整
ロール型	CrewAI, Agency Swarm	役割分担でスコープを限定
SOP（Standard Operating Procedures）型	MetaGPT	標準手順書でハルシネーション抑制
ファイル型	Agent Teams	各エージェントが自分の受けるプロンプトのみ参照

3.3. 各アプローチの特徴と適所

各アプローチの向いているケースと用途

- グラフ型: 状態管理が複雑・長期実行が必要な用途向き
- 会話型: 役割が動的に変わる向いているケース
- ロール型: チーム分業・高速プロトタイピングの用途向き
- SOP 型: 品質保証・再現性が求められる用途向き
- ファイル型: 透明性・人間介入が重要な向いているケース

どれが「正解」ではなく、用途・チーム・優先事項によって選ぶ

3.4. Anthropic が定義する 5 つのコアパターン

Anthropic “Building Effective Agents” (2024) — マルチエージェント設計の基本パターン

パターン	概要
Prompt Chaining	複数の LLM 呼び出しを連鎖させ、前の出力を次の入力にする
Routing	入力を分類し、それぞれ専門化されたサブタスクへ振り分ける
Parallelization	独立したタスクを複数エージェントで同時処理し、速度と品質を向上させる
Orchestrator-Workers	orchestrator が計画・委譲し、worker が実行する役割分担
Evaluator-Optimizer	生成担当と評価担当を分離し、フィードバックループで品質を高める

4. マルチエージェントの選択肢

4.1. 組み込みモードとトレードオフ

一般的なアプローチとそれぞれの特徴

アプローチ	概要	メリット	デメリット
subagent	Claude Code がサブタスクを委譲。各サブエージェントは独自コンテキストウインドウを持つ。`isolation: worktree` で作業ブランチを隔離可能	セットアップ不要・最大 10 並列実行可能	サブエージェント同士は通信不可。親のみに報告
skill	スキル定義に `context: fork` を追記するとメイン会話から隔離してサブエージェント内で実行される	メインコンテキストを汚染しない・一部スキルは並列実行	スキル間の通信なし。結果はメイン会話に戻るのみ
agent teams	Claude Code の実験的マルチエージェント機能（デフォルト無効・要有効化）。チームメートは互いに直接通信可能	チームメート間の双方向通信可	実験的・デフォルト無効・トークン消費が大幅に増加（Plan mode で約 7 倍）

出典: https://code.claude.com/docs/en/agent-teams / https://code.claude.com/docs/en/sub-agents / https://code.claude.com/docs/en/skills

4.2. 「Agent Teams で十分では？」という問いに答える

問い: Claude Code の Agent Teams が使えるなら、自前でオーケストレーションを組む必要はないのでは？

答え: ケースによる

Agent Teams で十分なケース:

- Claude Code ベースで完結する
- 実験的機能が許容できる環境
- セットアップコストを最小化したい

自前オーケストレーションが活きるケース:

- 異種 LLM を混在させたい（Codex CLI + Claude Code など）
- 実験的機能に依存したくない本番環境
- 人間がいつでも介入・観測できる透明性が必要
- トークンコストを抑えたい（Agent Teams は大幅なトークン増加を伴う）

重要: どちらが優れているではなく、用途に応じた選択の問題

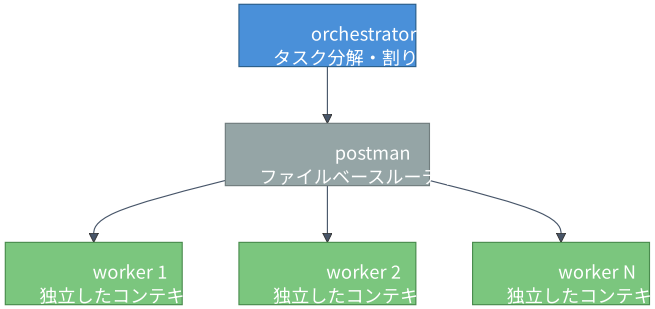
Anthropic の推奨原則: 「まず最もシンプルな解を見つけ、複雑性は効果が実証されたときのみ追加する」

5. 並列エージェントアプローチ

5.1. なぜ独立性が重要か

複数の独立したエージェントを並列に動かす考え方

- 各エージェントは自分のコンテキストだけを持つ
- 担当範囲を絞ることで、集中して高品質な出力を出しやすい
- 互いに干渉しないため、並列実行が自然にできる



前述の研究によると (arXiv:2511.15755, 348 回の対照実験)

- マルチエージェント: **100%** のアクション可能な推奨率（実際に実行できる具体的な提案の割合）
- 単一エージェント: **1.7%** のアクション可能な推奨率
- → 提案の具体性（行動特異性）で **80 倍**、解決策の正確性で **140 倍** の改善
- 348 件の対照実験で一貫した結果。プロンプトを専門化するほど、曖昧な提案が減る

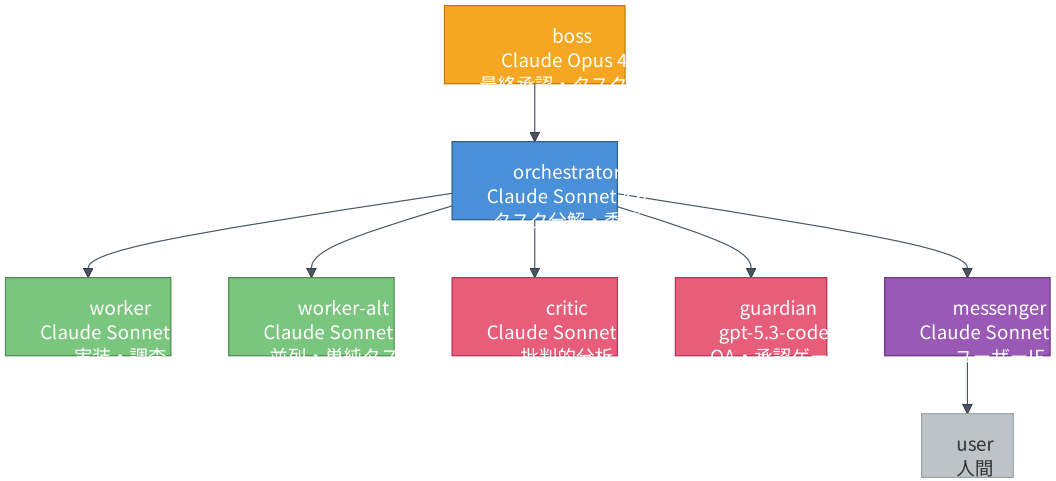
6. ロール設計

6.1. boss vs worker モデル

役割ごとに適したモデルを使い分ける

ポイント: 全員が同じモデルを使う必要はない。guardian には GPT 系モデルを使っており、異種 LLM 混在の実例でもある

研究による裏付け: FrugalGPT, RouterBench などが示すとおり、タスクの難易度に応じた異種モデルルーティングがコスト最適解



6.2. 組織設計パターンをモデルにする

人間の組織は、AI エージェントが直面する制約と同じ問題を解決するために進化してきた

- **認知的制限** → コンテキストウインドウ制限
- **タスク管理の限界** → 単一エージェントの処理能力限界
- **調整コスト** → エージェント間通信オーバーヘッド

なぜ組織構造が有効か

- boss, worker, critic, guardian というロールは、マネージャー・IC・QA・レビュアーという実績ある組織構造を反映
- **永続的な名前付きエージェント**は、一時的なサブエージェントより認知的に扱いやすい
- 結果として、**オペレーターの認知負荷が下がる**

Evaluator-Optimizer パターン

Anthropic の “Building Effective Agents” における 5 つのコアパターンの一つ: 一方の LLM が生成し、もう一方が評価・フィードバックするループ。critic + guardian ペアはこのパターンの評価器（evaluator）側の実装例

設計原則: AI エージェントシステムを機能的な人間の組織図に倣って設計する

7. 混在 LLM エコシステム

7.1. 宣言的ワークフローとは

ファイルベースのプロトコルなので、LLM の種類に依存しない

- Claude Code（Anthropic）と Codex CLI（OpenAI）が同一ワークフローに参加できる
- 各エージェントは自分の inbox ファイルを読むだけ
- エージェント間のプロトコルは YAML フロントマター + Markdown

「宣言的」とはどういう意味か

- 「誰が誰に話せるか」を設定ファイルに書くだけでルーティングを自動処理する
- **命令的**: メッセージごとに送り先を手動でコード化する必要がある
- **宣言的**: 新しいエージェントを追加するとき、edges 設定に 1 行加えるだけでよい

7.2. Anthropic 分類との対応

Anthropic は “Building Effective Agents” (2024) でシステム種別を 2 つに分類する

- **ワークフロー**: 事前定義されたコードパスで LLM/ツールを制御するシステム
- **エージェント**: LLM が自律的にプロセスを制御・ツール使用を判断するシステム

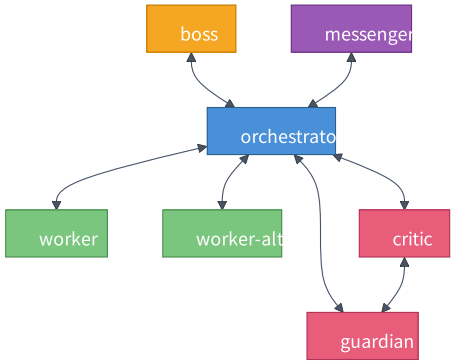
このシステム（tmux-a2a-postman）は **「ワークフロー」** に分類される

- postman.toml の edges で通信経路が事前定義されている
- LLM が自律的にルーティングを変更することはない
- 予測可能・検査可能・人間が介入しやすい

7.3. 設定例

宣言的エッジ設定の例（postman.toml）

```toml
[postman]
edges = [
  "boss -- orchestrator",
  "messenger -- orchestrator",
  "orchestrator -- worker",
  "orchestrator -- worker-alt",
  "orchestrator -- critic",
  "orchestrator -- guardian",
  "critic -- guardian",
]
```



この 6 エッジが「誰が誰に話せるか」をすべて定義する

8. Human-in-the-loop

8.1. 人間が介入できる仕組み

人間が手綱を握る — それが tmux ベースの最大の強みだ

組み込みモードの弱点

- エージェント間の通信が不透明になりがち
- 途中介入のタイミングが限られる
- ログが埋もれて追跡しにくい

tmux ベースの強み

- 各エージェントの出力を tmux ペインで即時確認し、品質をその場で改善する
- 全メッセージはファイルに記録され、後から参照する
- 人間が `boss` ロールを直接担い、出力品質をリアルタイムで引き上げる

8.2. 半自律改善サイクル

動かすほど、システムが育つ

- **全指示がパイプラインを通る** — critic → guardian → boss のレビューを経た内容がフィードバックされる。人間が介入しなくても品質が担保される
- **LLM の癖が可視化される** — 動かすほど、各 LLM の判断パターン・バイアスが見えてくる
- **意思決定の覗き見が糧になる** — 全メッセージはファイルに残り、なぜその判断をしたかを後から追える。これが設計改善の源泉

「半自律」である理由

改善サイクル自体を skill（専門エージェントへの指示書）として定義しておくことで、人間はトリガーを引くだけでよい — 完全自律ではないが:

1. 人間: 問題に気づいて skill を呼び出す
2. エージェント: skill の手順に従い、問題を特定・修正案を提案
3. 人間: 承認
4. エージェント: 修正を適用・検証

チューニングのコストをエージェントが肩代わりしてくれる構造

9. tmux-a2a-postman

9.1. tmux-a2a-postman とは

ファイルベースのメッセージングとルーティングを担う OSS

- GitHub: https://github.com/i9wa4/tmux-a2a-postman

tmux-a2a-postman は、各エージェントのペインタイトルをノード名として自動検出し、inbox/ ディレクトリへのファイル配送でエージェント間通信を実現する

主な特徴

- ペインタイトル = ノード名: `tmux list-panes` でセッション内の全エージェントを自動検出
- ファイルベース通信: メッセージは `inbox/`, `draft/`, `post/`, `read/` のディレクトリ構造で管理
- LLM 非依存: Claude Code, Codex CLI, 任意の CLI エージェントを混在可能
- 人間が `boss` ロールを担い、リアルタイムで品質を管理できる

tmux-a2a-postman を使うことで、tmux の独立プロセスアプローチに標準的なメッセージルーティング層を追加できる

10. まとめ

10.1. まとめ

今日のキーポイント

テーマ	要点
コンテキスト制約	研究でも確認済み。分割・専門化が品質を最大 80 倍改善
エコシステム	LangGraph, AutoGen, CrewAI など多様。問題は共通、解法は様々
ロール分担	役割ごとに適切なモデルを選ぶことでコスト最適化
組織設計パターン	人間の組織構造をモデルにすると設計しやすく運用しやすい
宣言的ワークフロー	edges 設定 1 行でエージェント追加。自動ルーティング
Human-in-the-loop	tmux で透明性を保ち、いつでも人間が介入できる

10.2. 次の一歩

もし試してみたいなら

1. 何か 1 つのフレームワークを試す
2. tmux に興味が出たら基本操作を習得（前回記事を参照）
3. tmux ベースが合いそうなら、orchestrator + worker の 2 ロールで小さく始める
4. 実際の業務タスクを 1 つ持ち込んで試してみる

重要な前提: どのアプローチも「コンテキストを小さく保つ」という原則は共通。ツールを選ぶ前にその原則を理解することが一番大切。